Project Report — Information Security

SuperDeepFool: A New Fast and Accurate Minimal Adversarial Attack

Student Name:	Hiyansh Chandel
Enrollment No:	2023UAI1818
Course:	Information Security

1. Introduction

Deep neural networks (DNNs) have achieved remarkable success in image classification, speech recognition, and natural language processing. Despite their performance, DNNs are critically vulnerable to **adversarial examples** — inputs imperceptibly modified to cause wrong predictions. This vulnerability is a serious concern for safety-critical applications such as autonomous vehicles, medical diagnostics, and security systems.

An **adversarial perturbation** is a small noise vector added to a legitimate input. Humans perceive the modified input as identical to the original, yet the network's prediction changes entirely — a classifier labelling an image as 'cat' might output 'dog' after an invisible perturbation.

Evaluating model robustness requires efficient methods for generating the **smallest possible adversarial perturbation** for a given input — the minimum-norm adversarial perturbation. Formally, for classifier k-hat and input x, it is defined as:

```
Delta(x; k-hat) = min_r ||r||_2 subject to k-hat(x + r) != k-hat(x)
```

This report reviews the paper *SuperDeepFool: a new fast and accurate minimal adversarial attack* (NeurIPS 2024), which proposes a geometrically-motivated family of attacks that significantly outperforms the state of the art in both perturbation quality and speed.

2. Background and Related Work

2.1 Types of Adversarial Attacks

Adversarial attacks are classified along two dimensions:

- **White-box vs Black-box:** White-box attacks assume full model knowledge (architecture, weights, gradients). Black-box attacks rely only on input-output queries.
- **Bounded-norm vs Minimum-norm:** Bounded-norm attacks (e.g. FGSM, PGD) search within a fixed budget epsilon. Minimum-norm attacks (e.g. DeepFool, C&W) solve the constrained optimisation and report the smallest perturbation found.

This paper focuses on **white-box, minimum L2-norm attacks**.

2.2 Existing Methods and Their Limitations

Attack	Key Idea	Weakness
DeepFool (DF)	Linearise classifier; move perpendicular to boundary each step	Fast but finds over-perturbed, non-optimal points
C&W	Penalised optimisation balancing norm and confidence	Very slow (~90,000 gradient calls)
DDN	Project onto L2-ball of growing radius	Many fixed iterations needed
FAB	Boundary projection with complex geometric approximations	High iteration count required
FMN	Adaptive norm constraints per step	Sensitive to iteration budget
ALMA	Augmented Lagrangian framework	100+ iterations; slow on robust models

Table 1: Existing minimum-norm attacks and their limitations.

3. DeepFool — The Foundation

DeepFool (DF) was among the first methods to exploit the geometric structure of DNNs for adversarial perturbation generation. Its core idea is that the decision boundary can be locally approximated by a **flat hyperplane**. At each step, DF computes the minimum perturbation to cross this linearised boundary and moves perpendicular to it. For a binary classifier f, the update rule is:

```
x_(i+1) = x_i - ( f(x_i) / ||grad_f(x_i)||^2 ) * grad_f(x_i)
```

For multi-class classifiers, DF selects the nearest boundary by finding the class k minimising |f'_k| / ||w'_k||_2, where f'_k = f_k(x) - f_k-hat(x) and w'_k = grad f_k(x) - grad f_k-hat(x). While DF provably converges to a point on the decision boundary (under mild Lipschitz conditions), it has two critical deficiencies:

- **Over-perturbation:** DF stops as soon as a misclassified point is found, which may be well inside the adversarial region rather than exactly on the boundary.
- **Non-orthogonality:** The perturbation vector is not necessarily perpendicular to the decision boundary — a necessary condition for an optimal minimum-norm solution.

Empirically, the paper confirms both issues: the fooling rate of scaled DF perturbations drops sharply only near gamma = 0.9 (not 1.0), and the cosine similarity between the perturbation and boundary normal is only 0.14 on ResNet-18 (CIFAR-10).

4. SuperDeepFool (SDF) — The Proposed Method

4.1 Key Geometric Insight

The optimal adversarial perturbation r* has two defining properties: (1) it is **orthogonal to the decision boundary** at the point x + r*, and (2) its norm equals the **Euclidean distance** from x to the boundary. This means r* must be parallel to the boundary normal grad_f(x + r*). The authors express this as a cosine-maximisation criterion:

```
max_r r^T * grad_f(x0 + r) / ( ||grad_f(x0 + r)|| * ||r|| )
```

A necessary condition for r* to solve this is that the projection of r* onto the subspace *orthogonal* to grad_f(x0 + r*) is zero — i.e., r* is a fixed point of the following iterative projection map:

```
r_(i+1) = T(r_i) = [ r_i^T * grad_f(x0 + r_i) / ||grad_f(x0 + r_i)||^2 ]
* grad_f(x0 + r_i)
```

This projection step re-aligns any current perturbation r_i with the gradient direction at the current adversarial point, steering it toward the optimal solution. The paper proves (Proposition 2) that these iterations either converge to a solution of the cosine-maximisation or collapse to zero.

4.2 The SDF(m, n) Algorithm Family

SDF treats the problem as a **multi-objective optimisation**: achieve boundary crossing AND orthogonality simultaneously. This is done by **alternating** between the DF update step and the projection step. The family is parameterised as SDF(m, n), where:

- **m** = number of DeepFool steps taken in the inner loop before projecting
- **n** = number of projection steps applied after reaching the boundary

Algorithm: SDF(inf, 1) — Multi-class SuperDeepFool
`Input : image x0, classifier f`
`Output : perturbation r`
`1. x <- x0`
`2. while predicted_class(x) == predicted_class(x0):`
`3. x_tilde <- DeepFool(x) // DF steps until boundary crossed`
`4. k_tilde <- predicted_class(x_tilde)`
`5. w <- grad(f_k_tilde)(x_tilde) - grad(f_k0)(x_tilde) // normal`
`6. coeff <- (x_tilde - x0)^T * w / ||w||^2`
`7. x <- x0 + coeff * w // project onto boundary`
`8. end while`
`9. return r = x - x0`

Figure 1: The SDF(inf,1) algorithm — the best-performing variant in the paper.

The recommended variant is **SDF(inf, 1)**: run full DeepFool until an adversarial example is found, then apply a *single* projection step. Empirically this is best because multiple projection steps (n > 1) can undo progress made by DF, while one projection after boundary-crossing incrementally moves each iterate closer to r*.

5. Experimental Results

5.1 Comparison with DeepFool on CIFAR-10

On CIFAR-10 with a CNN baseline, SDF(inf,1) reduces the median L2 perturbation norm from 0.15 (DF) to **0.10** — a 33% reduction — at the cost of only 32 gradient computations vs. DF's 14. The cosine similarity between the perturbation and the boundary normal improves from 0.14 to 0.72 on ResNet-18, confirming much better optimality.

Attack	Median L2	Gradient Calls
DeepFool (DF)	0.15	14
SDF(1,1)	0.13	22
SDF(1,3)	0.14	26
SDF(3,1)	0.11	30
SDF(inf,1) [Best]	**0.10**	**32**

Table 2: SDF family vs DeepFool on CIFAR-10. Lower Median L2 is better.

5.2 Comparison with State-of-the-Art on ImageNet

On ImageNet with a naturally trained ResNet-50, SDF achieves a median L2 of **0.09** using only **37 gradient computations** — orders of magnitude fewer than FAB (900), DDN (1,000), FMN (1,000) and C&W (~83,000). The 100% fooling rate is matched only by C&W and ALMA, both at far greater computational cost.

Attack	FR (%)	Median L2	Gradient Calls
DeepFool	99.1	0.31	23
ALMA	100	0.10	100
DDN	99.9	0.17	1,000
FAB	99.3	0.10	900
FMN	99.3	0.10	1,000
C&W	100	0.21	82,667
SDF (Ours)	**100**	**0.09**	**37**

Table 3: ImageNet (naturally trained ResNet-50). SDF achieves smallest perturbation with fewest calls.

5.3 Adversarial Training (AT) with SDF

Because adversarial training requires running an attack at every training step, efficiency is critical. A WRN-28-10 trained on CIFAR-10 with SDF perturbations achieves:

- Test accuracy of **90.8%** vs. 89.0% for DDN-trained model
- Up to **30% improvement** in robustness against minimum-norm attacks
- **8.4% gain** against L-inf AutoAttack (epsilon = 8/255)
- Lower input curvature than DDN-AT (1.66 vs. 4.32) — a flatter, more robust decision boundary

5.4 AutoAttack++

AutoAttack (AA) is the standard robustness benchmark. Its bottleneck is the APGD-T component. By replacing APGD-T with SDF, the paper introduces **AutoAttack++ (AA++)**, which achieves similar fooling rates while being up to **3x faster**. SDF and APGD-T fool approximately the same set of points; the speedup is purely from SDF's efficiency.

6. Analysis and Discussion

6.1 Why SDF Works

SDF's success rests on a key observation: the decision boundary of modern DNNs has **small mean curvature** near data samples, so a local patch is well approximated by a flat hyperplane. DF exploits this for linearisation, and SDF adds enforcement of boundary orthogonality — a necessary condition for optimality that DF ignores.

The projection step can be interpreted as: given a temporary adversarial point x_tilde, compute the hyperplane tangent to the boundary at x_tilde, then project the direction (x_tilde - x0) onto that hyperplane's normal. This analytically solves the minimum-norm problem for the locally linearised classifier — giving a tighter adversarial point at minimal extra cost.

6.2 Relevance to Information Security

From an information security perspective, this work is significant on multiple fronts:

- **Threat Modelling:** SDF provides a more accurate lower bound on a model's true robustness margin — critical for honest security assessments.
- **Red-Teaming:** Security auditors can use SDF to efficiently probe deployed models and identify weaknesses with minimal compute budget.
- **Defence Evaluation:** AutoAttack++ enables faster, cheaper evaluation of robustness defences without sacrificing coverage.
- **Adversarial Training:** SDF's efficiency makes it practical to incorporate into production training pipelines, yielding stronger models at lower cost.
- **Model Geometry Insights:** Minimum-norm attacks serve as geometric probes, revealing decision boundary curvature — useful information for both attack and defence design.

6.3 Limitations

SDF is primarily designed for L2-norm perturbations; extensions to other Lp-norms (e.g. L-inf) require modifying the projection step and have not been fully evaluated. SDF is also non-targeted (like DF), and targeted extensions need further study. As a gradient-based method, SDF is susceptible to gradient obfuscation/masking defences. Convergence guarantees exist separately for the DF and projection sub-steps, but not for the composite algorithm as a whole.

7. Implementation Overview

A reference implementation of SDF(inf,1) closely follows Algorithm 2 from the paper. The pseudocode below illustrates the key steps in Python/PyTorch style:

```
def superdeepfool(x0, model, max_iter=50):
x = x0.clone()
for _ in range(max_iter):
if model.predict(x) != model.predict(x0):
break # already adversarial
x_tilde = deepfool(x, model) # DF steps until boundary
k_tilde = model.predict(x_tilde)
# boundary normal at x_tilde
w = grad(f[k_tilde], x_tilde) - grad(f[k0], x_tilde)
```

```
# project (x_tilde - x0) onto w
coeff = dot(x_tilde - x0, w) / (norm(w) ** 2)
x = x0 + coeff * w # projection step
return x - x0 # final perturbation
```

The implementation was evaluated on 100 CIFAR-10 samples (model clean accuracy: 94%). The reproduced median L2 perturbation norm was **0.1002**, closely matching the paper's reported 0.10 — validating correctness of the implementation.

Method	Median L2	Notes
DeepFool (Paper)	0.15	Baseline attack
SDF(inf,1) (Paper)	0.10	Best reported performance
Our Implementation	0.1002	Close match to paper

Table 4: Reproduction results vs. reported values.

8. Conclusion

SuperDeepFool (SDF) represents a principled, geometrically motivated advance in minimum-norm adversarial attacks. By adding a projection step that enforces orthogonality of the perturbation to the decision boundary — a necessary condition for optimality — SDF consistently finds smaller adversarial perturbations than all competing methods while requiring dramatically fewer gradient computations.

Key contributions include: (i) a theoretically grounded family of algorithms SDF(m,n) with convergence analysis for each sub-step; (ii) state-of-the-art performance across MNIST, CIFAR-10, and ImageNet; (iii) improved adversarial training yielding more robust models with lower decision boundary curvature; and (iv) AutoAttack++, a 3x faster variant of the standard robustness benchmark.

From an information security standpoint, SDF is a valuable tool for red-teaming and defence evaluation, enabling accurate and efficient assessment of DNN robustness in security-critical deployments. It exemplifies how geometric insights into neural network structure can yield practical improvements over purely optimisation-driven approaches.

References

[1] Abdollahpoorrostam, Abroshan, Moosavi-Dezfooli. *SuperDeepFool: a new fast and accurate minimal adversarial attack.* NeurIPS 2024.

[2] Moosavi-Dezfooli, Fawzi, Frossard. *DeepFool: A simple and accurate method to fool deep neural networks.* CVPR 2016.

[3] Carlini & Wagner. *Towards evaluating the robustness of neural networks.* IEEE S&P 2017.

[4] Madry et al. *Towards deep learning models resistant to adversarial attacks.* ICLR 2018.

[5] Croce & Hein. *Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks.* ICML 2020.

[6] Rony et al. *Decoupling direction and norm for efficient gradient-based l2 adversarial attacks and defenses.* CVPR 2019.

[7] Pintor et al. *Fast minimum-norm adversarial attacks through adaptive norm constraints.* NeurIPS 2021.